DAM | DISEÑO DE INTERFACES | UNIDAD 2 • PARTE 1/2

CheatSheet #01: Ejercicios 1 y 2 (WPF & XAML)

Repaso exprés de examen • Layouts sin marco, DockPanel, MetroWindow y Recursos

IES Henri Matisse
2º DAM • Evaluación Práctica

EJERCICIO 01 SafetyBoost Dashboard (Ventana Sin Marco & DockPanel)

DI.UNIDAD2.EJERCICIO1

🎯 **CONCEPTO CLAVE:** Ventana moderna sin marco del SO (`WindowStyle="None"`) con layout híbrido `DockPanel` + `Grid` anidados y control manual de ventana en C# (`WindowState` y `Close()`).

Resumen de estudio: repasa la estructura del layout, la necesidad de una barra de control propia y la diferencia entre `Grid.Column` y `Grid.ColumnSpan` para que la composición no se rompa en el examen.

1. Ventana Sin Bordes
`WindowStyle="None"` y `AllowsTransparency="True"` eliminan la barra nativa del SO.

2. Acoplamiento DockPanel
Ancla panel lateral a la izquierda (`Dock="Left"`) y barra roja arriba (`Dock="Top"`).

3. Rejilla Central Responsiva
`Grid` con columnas/filas proporcionales (*) y expansión con `ColumnSpan`.

SNIPPET INTERFAZ CRUCIAL (MAINWINDOW.XAML) — XAML

```xaml
<!-- Ventana sin marco nativo y maximizada -->
<Window x:Class="di.unidad2.ejercicio1.MainWindow"
        WindowStyle="None" AllowsTransparency="True"
        WindowState="Maximized">

    <DockPanel>
        <!-- Lateral izquierdo anclado: ancho fijo -->
        <StackPanel DockPanel.Dock="Left" Width="200" Background="#333">
            <Image Source="/recursos/iconos/TV.png" Width="64" />
            <Button Background="Transparent" BorderBrush="Black">
                <StackPanel Orientation="Horizontal">
                    <Image Source="/recursos/iconos/LineChart.png" Width
                    <TextBlock Text="stats" Foreground="White" Margin="3
                </StackPanel>
            </Button>
        </StackPanel>

        <!-- Barra superior: Rejilla con botones de ventana -->
        <Grid DockPanel.Dock="Top" Height="60" Background="#e8464d">
            <Grid.ColumnDefinitions>
                <ColumnDefinition Width="Auto" /> <!-- Título -->
                <ColumnDefinition Width="Auto" /> <!-- Subtítulo -->
                <ColumnDefinition Width="Auto" /> <!-- Acción -->
                <ColumnDefinition Width="*" />    <!-- Espaciador + mini
                <ColumnDefinition Width="Auto" /> <!-- Maximizar -->
                <ColumnDefinition Width="Auto" /> <!-- Cerrar -->
            </Grid.ColumnDefinitions>
            <Button Grid.Column="3" Click="btnminimice_click">
                <Image Source="/recursos/iconos/icons8-minimize-32.png"
            </Button>
            <Button Grid.Column="4" Click="btnmaximice_click">
              <Image Source="/recursos/iconos/icons8-maximize-50.png" />
            </Button>
            <Button Grid.Column="5" Click="btnClose_click">
                <Image Source="/recursos/iconos/icons8-close-50.png" />
            </Button>
        </Grid>

        <!-- Área central: ocupa el espacio restante -->
        <Grid>
            <Image Grid.Column="1" Grid.ColumnSpan="2" Source="/Recursos
            <Image Grid.Row="1" Grid.ColumnSpan="3" Stretch="Fill" Sourc
        </Grid>
    </DockPanel>
</Window>
```

SNIPPET LÓGICA DETRÁS (MAINWINDOW.XAML.CS) — C#

```csharp
namespace di.unidad2.ejercicio1
{
    // Herencia estándar : Window
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            // ¡Imprescindible! Enlaza XAML con C#
            InitializeComponent();
        }

        // 1. Cerrar aplicación
        private void btnClose_click(object sender, Route
        {
            this.Close();
        }

        // 2. Minimizar a la barra de tareas
        private void btnminimice_click(object sender, Ro
        {
            this.WindowState = WindowState.Minimized;
        }

        // 3. Alternar Maximizar ⟷ Normal
        private void btnmaximice_click(object sender, Ro
        {
            if (this.WindowState == WindowState.Maximize
                this.WindowState = WindowState.Normal;
            else
                this.WindowState = WindowState.Maximized
        }
    }
}
```

⚠️ POSIBLES TRAMPAS DE EXAMEN (EJERCICIO 1)

- ✕ **Ventana Bloqueada sin Salida:** Al usar `WindowStyle="None"` desaparece el botón de cerrar del SO. Si no programas `this.Close()` en un botón, la ventana queda atrapada en pantalla completa.
- ✕ **Olvidar `InitializeComponent()`:** Si se omite o se interactúa con controles antes de llamarlo en el constructor, se produce un `NullReferenceException` inmediato al iniciar.
- ✕ **Confusión `Column` vs `ColumnSpan`:** `Grid.Column="1"` ubica el control en la 2ª columna. Para que ocupe varias columnas consecutivas es obligatorio usar `Grid.ColumnSpan="2"`.
- ✕ **Orden de declaración en `DockPanel`:** En WPF el último hijo del `DockPanel` rellena todo el espacio central por defecto (`LastChildFill="True"`). Colocar el área central antes que los laterales arruina el layout.

CheatSheet #01: Ejercicios 1 y 2 (WPF & XAML)

Repaso exprés de examen • MahApps.Metro, Material Design, PasswordBox y MergedDictionaries

IES Henri Matisse
2º DAM • Evaluación Práctica

EJERCICIO 02 Login "Welcome Folk" (MahApps.Metro & Material Design)

DI.UNIDAD2.EJERCICIO2

🎯 **CONCEPTO CLAVE:** Integración de librerías visuales NuGet (`MahApps.Metro` y `MaterialDesign`), fusión global de recursos en `App.xaml` (`MergedDictionaries`) y herencia estricta de `MetroWindow` en C#.

Resumen de estudio: el diagnóstico rápido es comprobar que el tema visual está cargado en `App.xaml`, que la ventana usa `MetroWindow` y que la contraseña se maneja con `PasswordBox`.

1. Diccionarios Globales

`App.xaml` fusiona estilos de controles, fuentes y paleta de color (`Dark.Cobalt.xaml`).

2. MetroWindow en XAML y C#

Ventana plana moderna; requiere coincidencia de herencia en XAML y en el code-behind C#.

3. Seguridad & Vectores

Campo seguro `PasswordBox` e iconos vectoriales de Material Design (`mat:PackIcon`).

🎨 EXPLICACIÓN VISUAL DEL EJERCICIO 2

XAML

```xaml
<!-- La clave está en la estructura del formulario: imagen + panel de lo
<mah:MetroWindow Title="LOGIN" WindowStartupLocation="CenterScreen"
    xmlns:mah="http://metro.mahapps.com/winfx/xaml/controls"
    xmlns:mat="http://materialdesigninxaml.net/winfx/xaml/themes">

    <Grid Background="#04113e">
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="5*" />
            <ColumnDefinition Width="5*" />
        </Grid.ColumnDefinitions>

        <Image Source="/recursos/imagen/login.png" Stretch="UniformToFil

        <StackPanel Grid.Column="1" VerticalAlignment="Center">
            <mat:PackIcon Kind="Google" Width="42" Height="42" />
            <TextBox x:Name="txtEmail" Foreground="White" />
            <PasswordBox x:Name="txtPassword" Foreground="White" />
        </StackPanel>
    </Grid>
</mah:MetroWindow>
```

🧠 QUÉ DEBES ENTENDER BIEN

EXAMEN

```
1. App.xaml debe cargar los estilos globales de MahApps
2. El control principal de la ventana es un MetroWindow,
3. La parte izquierda suele ser una imagen o ilustración
4. Los iconos material se usan con PackIcon, no con imág
5. PasswordBox protege la contraseña y no debe tratarse
6. El objetivo es estética + organización visual, no lóg

Este ejercicio no exige explicar el code-behind.
```

⚠️ POSIBLES TRAMPAS DE EXAMEN (EJERCICIO 2)

- ✕ **Olvidar los diccionarios globales:** Si no fusionas los estilos de `MahApps.Metro` y `MaterialDesign` en `App.xaml`, el programa se verá roto o fallará al arrancar.
- ✕ **Mezclar layout vertical y columnas:** Si la imagen y el formulario se dejas sin una estructura clara en `Grid`, la vista se descuadra y pierdes la composición del login.
- ✕ **Inventar lógica extra:** No hace falta crear validaciones complejas ni autenticar usuarios si la práctica no lo pide; lo importante es el diseño y la composición.
- ✕ **Usar `TextBox` para la contraseña:** La contraseña se recoge en `PasswordBox` y su lectura se hace con `.Password`, no con `.Text`.
- ✕ **Importar iconos a mano cuando hay PackIcon:** Para este tipo de ejercicios suele ser más correcto usar `mat:PackIcon` porque es más limpio y visualmente consistente.